SR UNIVERSITY

School of Computer Science and Artificial Intelligence

Lab Assignment # 5.2

Program	**: B. Tech (CSE)**
Specialization	**: –**
Course Title	**: AI Assisted coding**
Course Code	**:**
Semester	**II**
Academic Session	**: 2025-2026**
Name of Student	**: Akash Reddy**
Enrollment No.	**: 2403A51L30**
Batch No.	**: 51**
Date	**:13-01-2026**

LAB ASSIGNMENT - 5.2

TASK 1: AI-Generated Logic Without Modularization (Factorial without Functions)
Prompt : Write a Python program to calculate factorial of a number using loops only.
Do not use functions.
Code:

```
# Factorial calculation without using functions

n = int(input("Enter a number: "))

factorial = 1

if n < 0:
    print("Factorial is not defined for negative numbers")
else:
    for i in range(1, n + 1):
        factorial = factorial * i

    print("Factorial of", n, "is", factorial)
```

Output:

```
Factorial of 5 is 120
```

Explanation:
GitHub Copilot was very helpful in quickly generating correct and readable code for a beginner. It followed basic coding practices automatically and reduced development time. However, manual review is still necessary to ensure optimal design and efficiency.

TASK 2: AI-Generated Code (Without Functions)

Prompt : Optimize this code, reduce unnecessary variables and improve readability.

Code:

```
#Optimize this code, reduce unnecessary variables and improve readability.

n = 5
if n < 0:
    print("Factorial is not defined for negative numbers")
else:
    result = 1
    for i in range(1, n + 1):
        result *= i
    print(f"Factorial of {n} is {result}")
```

Output:

```
Factorial of 5 is 120
```

Explanation :

- The optimized code improves readability using f-strings and simpler variable names.
- Unnecessary blank lines were removed.
  - The logic remains the same, but the code is cleaner and easier to maintain.

Task 3: Modular Design Using AI Assistance(Factorial with Functions)

Prompt : Create a Python program to calculate factorial using a function with comments.

Code:

```
# Function to calculate factorial of a number
def calculate_factorial(number):
    result = 1
    for i in range(1, number + 1):
        result *= i
    return result

# Main execution block
num = 5

if num < 0:
    print("Factorial is not defined for negative numbers")
else:
    print(f"Factorial of {num} is {calculate_factorial(num)}")
```

Output :

```
Factorial of 5 is 120
```

Explanation:

- Using functions improves reusability because the same logic can be used in multiple programs.
- It makes the code easier to test, debug, and maintain.
- Modular design is essential for larger applications and team-based development.

Task 4: Code for Comparative Analysis

Prompt : 1. Procedural Code (Without Functions)

```
# Factorial program without using functions

n = 5

fact = 1

if n < 0:
    print("Factorial is not defined for negative numbers")
else:
    for i in range(1, n + 1):
        fact *= i
    print("Factorial of", n, "is", fact)
```

Output:

```
Factorial of 5 is 120
```

Prompt 2: Modular Code (With Functions)

```
# Factorial program using a function

def factorial(number):
    result = 1
    for i in range(1, number + 1):
        result *= i
    return result

num = 5

if num < 0:
    print("Factorial is not defined for negative numbers")
else:
    print("Factorial of", num, "is", factorial(num))
```

Output:

```
Factorial of 5 is 120
```

Explanation :

The modular code is more readable and reusable because the logic is separated into a function. It is easier to debug and maintain, making it suitable for large projects. The non-modular code is

simpler but less flexible and not scalable

Task 5: AI-Generated Iterative vs Recursive Thinking

Prompt : Generate an iterative factorial program in Python.

```
def factorial_iterative(n):
    result = 1
    for i in range(1, n + 1):
        result *= i
    return result
```

Prompt : Generate a recursive factorial program in Python.

```
def factorial_recursive(n):
    if n == 0 or n == 1:
        return 1
    return n * factorial_recursive(n - 1)
```

Explanation:
The iterative approach calculates the factorial using a loop and is efficient with minimal memory usage. The recursive approach solves the problem by calling the function repeatedly until a base condition is met, which uses more stack memory. Recursion is not recommended for large inputs due to the risk of stack overflow.